Обработка прерываний.
Машино-зависимые свойства операционных систем

1. Понятие прерывания. Последовательность действий при обработке прерываний. Классы прерываний. Рабочая область прерываний. Вектор прерывания.
2. Стандартные программы обработки прерываний, драйверы устройств. Приоритеты прерываний. Вложенные прерывания.

Обработка прерываний

Прерывание (interrupt) – это сигнал, заставляющий ЭВМ менять обычный порядок выполнения команд процессором.

Возникновение подобных сигналов обусловлено такими *событиями*, как:

завершение операций ввода-вывода.
истечение заранее заданного интервала времени.
попытка деления на нуль.
сбой в работе аппаратного устройства и др.
Обработчик прерываний – программа обработки прерывания, являющаяся частью ОС, предназначенная для выполнения ответных действий на условие, вызвавшее прерывание.

Предположим, что в момент поступления сигнала прерывания от некоторого источника программа А находится в решении. В результате управление автоматически передается обработчику прерываний. После завершения обработки управление может быть снова передано в ту точку программы А, где ее выполнение было прервано.

Классификация прерываний
Классы прерываний
В зависимости от источника возникновения сигнала прерывания делятся на: Аппаратные (внешние, асинхронные) — события, которые исходят от внешних источников (например, периферийных устройств) и могут произойти в любой произвольный момент (принцип асинхронности): сигнал от таймера, сетевой карты или дискового накопителя, нажатие клавиш клавиатуры, движение мыши; Программные (внутренние) — инициируются исполнением специальной инструкции в коде программы. Программные прерывания как правило используются для обращения к функциям встроенного программного обеспечения (firmware), драйверов и операционной системы. вызываются искусственно с помощью соответствующей команды из программы (int), предназначены для выполнения некоторых действий операционной системы, являются синхронными. Исключения (внутренние) — события в самом процессоре как результат нарушения каких-то условий при исполнении машинного кода: деление на ноль или переполнение, обращение к недопустимым адресам или недопустимый код операции; Вторая классификация – по местоположению источника возникновения запроса: Внутренние Внешние Типы внешних прерываний Внешние прерывания возникают по сигналу какого-нибудь внешнего устройства. Иногда желательно сделать систему нечувствительной ко всем или отдельным аппаратным прерываниям (т. е. от внешних устройств). Для этого используют так называемое маскирование внешних прерываний. Внешние прерывания подразделяются на: не маскируемые (англ. Non maskable interrupt, NMI) — обрабатываются всегда, независимо от запретов на другие прерывания. К примеру, такое прерывание может вызвать сбой в микросхеме памяти маскируемые - которые можно запрещать установкой соответствующих битов в регистре маскирования прерываний (в

x86-процессорах — сбросом флага IF в регистре флагов); Не маскируемые прерывания (говорят, что оно одно, т. к. подается на вывод микропроцессора NMI) инициируют источники, требующие безотлагательного вмешательства со стороны микропроцессора. 6 Обработка прерываний Маскируемые прерывания генерируются контроллером прерываний по заявке определенных периферийных устройств. Контроллер прерываний (выполнен в виде специальной микросхемы i8259A) поддерживает восемь уровней (линий) приоритета; к каждому уровню “привязано” одно периферийное устройство. Именно маскируемые прерывания часто называют аппаратными прерываниями. В реальном и защищенном режиме работы микропроцессора обработка прерываний осуществляется принципиально разными методами. Существуют два специальных внешних сигнала среди входных сигналов процессора, при помощи которых можно прервать выполнение текущей программы и тем самым переключить работу центрального процессора. Это сигналы NMI (no mask interrupt, не маскируемое прерывание) и INTR (interrupt request, запрос на прерывание).
Программные (внутренние)
Прерывания Программы могут сами вызывать прерывания с заданным номером. Для этого они используют команду INT. Это так называемые программные прерывания. Программные прерывания не являются асинхронными, так как вызываются из программы. Программные прерывания удобно использовать для организации доступа к отдельным, общим для всех программ функциям. Например, функции операционной системы доступны прикладным программам именно через прерывания. При вызове этих модулей нет необходимости знать их текущий адрес в памяти. Прикладные программы и драйверы могут сами устанавливать свои обработчики прерываний для их последующего использования другими программами. Для этого встраиваемые обработчики прерываний должны быть резидентными в памяти. Исключения (внутренние прерывания) Эти прерывания создаются цепями самого процессора при возникновении одной из специально оговоренных ситуаций. Если процессор сталкивается с одной из таких сбойных ситуаций, то он выполняет процедуру прерывания, используя закрепленной за ней вектор прерывания , который известен процессору. Например : Прерывание 0 вырабатывается в случае переполнения при операции деления на 0 Аппаратные и программные средства системы прерываний Система прерываний - это совокупность программных и аппаратных средств, реализующих механизм прерываний. К аппаратным средствам системы прерываний относятся: 9 Обработка прерываний выводы микропроцессора - на них формируются сигналы, извещающие микропроцессор либо о том, что некоторое внешнее устройство «просит уделить ему внимание» (INTR, заявка на прерывание), либо о том, что требуется безотлагательная обработка некоторого события или катастрофическая ошибка (NMI) INTR - вывод для входного сигнала запроса на прерывание, NMI - вывод для входного сигнала не маскируемого прерывания INTA - вывод для выходного сигнала подтверждения получения сигнала прерывания микропроцессором; программируемый контроллер прерываний 8259A (предназначен для фиксирования сигналов прерываний от восьми различных внешних устройств; он выполнен в виде микросхемы; именно он формирует номер вектора прерывания и выдает его шину данных); внешние устройства (таймер, клавиатура, магнитные диски и т. п.) К программным средствам системы прерываний реального режима относятся: таблица векторов прерываний. Занимает первый килобайт ОП (адреса 00000h-003FFh). Она содержит адреса (векторы - «векторы», т. к. два значения для указания адреса) обработчиков прерываний и состоит из 256 (0..255) элементов по 4 байта каждый: - 2 байта - новое значение для регистра IP - 2 байта - новое значение для регистра CS. Расположение таблицы векторов прерываний в процессорах i80286 и старше определяется значением регистра IDTR. Таблица векторов прерываний инициализируется при запуске системы, но в принципе может быть изменена и перемещена. Каждый вектор имеет свой номер и называется номером прерывания. два флага в регистре флагов flags/eflags: IF (Interrupt Flag) - флаг прерывания. Предназначен для маскирования (запрещения) аппаратных прерываний. Если IF=1, микропроцессор обрабатывает внешние прерывания, если = 0, то

игнорирует; TF(Trace Flag) - флаг трассировки. Если он=1, то микропроцессор переходит в режим по командной работы. В этом режиме в микропроцессоре генерируется внутреннее прерывание с номером 1; машинные команды микропроцессора: int, into (прерывание по переполнению), iret, cli, sti

Приоритезация прерываний

До окончания обработки прерывания обычно устанавливается запрет на обработку этого типа прерывания, чтобы процессор не входил в цикл обработки одного прерывания. Приоритезация означает, что все источники прерываний делятся на классы и каждому классу назначается свой уровень приоритета запроса на прерывание. Т.е. процессор может вести себя по разному при поступлении более приоритетного запроса. Приоритеты могут обслуживаться как относительные и абсолютные. Относительное обслуживание прерываний означает, что если во время обработки прерывания поступает более приоритетное прерывание, то это прерывание будет обработано только после завершения текущей процедуры обработки прерывания. Абсолютное обслуживание прерываний означает, что если во время обработки прерывания поступает более приоритетное прерывание, то текущая процедура обработки прерывания вытесняется, и процессор начинает выполнять обработку вновь поступившего 11 Обработка прерываний более приоритетного прерывания. После завершения этой процедуры процессор возвращается к выполнению вытесненной процедуры обработки прерывания.

Выводы

Прерывание можно рассматривать как некоторое особое событие в системе, требующее моментальной реакции. Механизм прерываний отлично подходит для управления событиями в вычислительной системе. Система распознает прерывания и при их возникновении запускает процедуру, соответствующую номеру прерывания. Существуют зарезервированные для использования центральным процессором прерывания.

Машинно зависимые свойства операционных систем

Успешно изучив материал, вы **будете знать**:

- что такое процесс;
- что такое поток;
- что такое устройства ввода-вывода;
- что такое прерывание;
- что такое виртуальная память.

После изучения данной темы Вы **будете уметь**:

- определять тип устройства ввода-вывода;
- различать процессы и потоки;
- определять допустимый объем виртуальной памяти.

После изучения материала Вы **будете обладать навыками**:

- классификации устройств ввода-вывода по различным критериям;
- диагностики компонентов аппаратных платформ на предмет выявления и устранения конфликтов.

Основные понятия к теме 2

Процесс

Поток

Символьный режим

Блочный режим

Контроллер

Драйвер устройства

Прерывание

Обработчик прерываний

Виртуальная память

Страничная организация памяти

Файл подкачки

Процессы и потоки

Одним из важнейших понятий операционных систем является понятие процесса.

Процесс — программа, которая в данный момент выполняется вычислительной машиной.

Каждому процессу выделяется отдельный, изолированный от других, сегмент памяти, который называется **адресным пространством процесса** . В адресном пространстве процесса, кроме самого процесса, также хранятся входные и выходные данные процесса. В многозадачной операционной системе все процессы выполняются по очереди таким образом, что в каждый момент времени выполняется только один процесс. Для обеспечения корректной работы процессов необходимо отслеживать состояние каждого процесса, чтобы возобновлять его выполнение с того момента, где в последний раз процесс был остановлен.

Кроме состояния процесса необходимо также отслеживать информацию об используемых им ресурсах. Вся информация о процессах хранится в таблице процессов — **массиве структур данных**, записями которого является информация по каждому процессу, запущенному в системе. Таблица процессов представляется обычно в виде дерева, потому что большинство процессов, выполняемых на машине, способно порождать дочерние процессы для решения вспомогательных задач, и для каждого процесса необходимо учитывать не только его собственное состояние, но и состояние всех связанных с ним процессов. Каждому процессу назначается два идентификатора, первый из которых указывает на сам процесс, а второй — на процесс, его породивший. Традиционно при загрузке операционной системы запускается некий базовый процесс, который порождает все прочие процессы — как системные, так и пользовательские. Этот процесс реализует главную функцию операционной системы — контроль вычислительных ресурсов. В современных версиях Windows этот процесс называется «System Idle». В UNIX-подобных ОС первый запускаемый процесс называется «init».

Задача планирования процессов заключается в отслеживании их состояния и использования ими вычислительных ресурсов. Как уже упоминалось в теме 1, вычислительный ресурс в каждый конкретный момент времени может

быть задействован только одним процессом. Если несколько процессов должны использовать один и тот же ресурс, то они используют его по очереди. Очередность использования определяется приоритетом процесса. Чем выше приоритет процесса, тем чаще он будет получать доступ к требуемым ресурсам.

Каждый процесс представлен как минимум одним потоком.

Поток — последовательность исполняемых команд.

В рамках одного и того же процесса может выполняться несколько разных потоков. Использование нескольких потоков позволяет сократить время исполнения программы. Такой подход удобен, если этапы решения задачи, для которой создавалась программа, можно выполнять параллельно. Потоки обладают некоторыми свойствами процессов. В отличие от процессов, потоки существуют в одном и том же адресном пространстве и могут одновременно работать с выделенными процессу ресурсами.

Устройства ввода-вывода. Прерывания

Устройства ввода-вывода делятся на две категории — блочные и символьные.

1. **Блочные устройства** — оперируют блоками данных, размер которых варьируется в зависимости от устройства. Каждый блок в блочном устройстве имеет собственный адрес. Примером блочного устройства может служить любой накопитель. Одним из наиболее важных свойств блочного устройства является возможность независимого доступа к блокам данных.

2. **Символьные устройства** — оперируют потоками данных, не имеющими структуры или адреса. Большинство устройств являются символьными.

Устройство ввода-вывода обычно состоит из двух частей — само устройство и его контроллер. **Контроллер** осуществляет управление работой устройства на физическом уровне. Контроллер выполняется в виде набора микросхем и либо совмещен с устройством, либо установлен на системной плате. Если контроллер установлен на системной плате, то обычно он позволяет работать с двумя и более устройствами данного типа. Примером такого контроллера может служить контроллер накопителей на жестких магнитных дисках, который позволяет работать одновременно с двумя НЖМД. Задачей контроллера является преобразование потока битов в блок байтов. Считываемые биты накапливаются в памяти контроллера, которая называется буфером данных, и затем в виде блоков байтов передаются в оперативную память. Каждый контроллер, помимо буфера, имеет также несколько регистров, посредством которых процессор может управлять работой контроллера.

Существует два альтернативных **способа управления контроллерами устройств**.

Первый способ заключается в том, что каждому регистру назначается уникальный номер порта ввода-вывода. При таком способе адресное пространство оперативной памяти не пересекается с адресным пространством устройств ввода-вывода.

Второй способ заключается в выделении каждому регистру отдельного сегмента оперативной памяти. Преимущество второго способа в том, что для программирования работы устройств не нужно прибегать к машинным языкам, а также в том, что при таком подходе для защиты от несанкционированного доступа к устройствам достаточно исключить часть адресного пространства устройств ввода-вывода из блока адресов памяти, доступных пользователям. Недостаток этого подхода в том, что для его реализации необходимо использование более сложной аппаратуры. Повышение сложности аппаратуры обуславливается тем фактором, что некоторые приемы, используемые для ускорения работы с памятью, могут привести к катастрофическим последствиям при использовании их с устройствами ввода-вывода.

Для ускорения обмена данными между процессором и устройствами используется **механизм прямого доступа к памяти (DMA)**. Контроллер DMA обеспечивает чтение данных с устройства в память и запись данных из памяти на устройство. В общих чертах механизм DMA работает следующим образом: когда процессу требуется прочитать или записать данные на устройство ввода-вывода, процессор программирует контроллер DMA на выполнение необходимой операции, сообщая ему, с каким устройством необходимо выполнить требуемую операцию и по какому адресу в памяти размещаются данные. До тех пор пока контроллер выполняет операцию, процессор может выполнять другие процессы. Когда контроллер DMA завершает порученную ему задачу, он посылает процессору сигнал о том, что операция завершена, после чего процессор приостанавливает выполнение текущей задачи и начинает выполнять тот процесс, который затребовал ввод-вывод данных.

Прерывание — это сигнал процессору о том, что ему необходимо прервать выполнение текущего процесса и вызвать обработчик прерывания.

Обработчик прерывания — это программа, которую процессор должен выполнить при возникновении прерывания. Обработчик прерывания является частью драйвера устройства. **Драйвер устройства** обеспечивает взаимодействие устройства с операционной системой. В состав операционной системы включен набор стандартных драйверов для поддержки различного оборудования, которое может быть подключено к компьютеру. Если подключаемое устройство обладает какими-либо функциями, которые недоступны операционной системе при использовании стандартного драйвера для данного типа оборудования, то такое устройство, как правило, снабжено собственным драйвером, который устанавливается при первом использовании устройства.

2.3. Виртуальная память

Виртуальная память (swap) — такой метод работы с памятью, когда в оперативной памяти хранятся только те части программы, которые используются в конкретный момент времени.

Все прочие части программы, равно как и данные, хранятся на диске. Этот способ организации памяти позволяет выполнять программы, чей суммарный объем вместе с данными может превышать объем доступной физической памяти. Большинство систем виртуальной памяти используют механизм **страничной организации памяти**. Вся доступная память

разбивается на страничные блоки фиксированного объема. Место для хранения страниц на жестких дисках должно быть выделено заранее. Временное хранение страниц виртуальной памяти обеспечивает специальный файл, размещающийся на одном или нескольких дисках компьютера, — **файл подкачки**. При обращении к какой-либо ячейке памяти запрос сначала передается диспетчеру памяти, который преобразовывает виртуальный адрес в реальный и передает полученный адрес на шину, который затем обрабатывается надлежащим образом. Объем виртуальной памяти регулируется пользователем.

Основные выводы

1. Процесс — это программа, выполняемая на процессоре, плюс обрабатываемые данные.
2. Каждый процесс состоит как минимум из одного потока.
3. Задача, решаемая многопоточным процессом, решается быстрее, чем однопоточным.
4. Устройства ввода-вывода бывают символьные и блочные. Блочное устройство позволяет работать одновременно с несколькими блоками данных.
5. Для работы с устройствами ввода-вывода используются механизм прямого доступа к памяти и прерывания.
6. Драйвер устройства необходим операционной системе для того, чтобы осуществлять взаимодействие с данным видом устройств.
7. Механизм виртуальной памяти позволяет выполнять программы, которые требуют больше физической памяти, чем есть на компьютере.

Контрольные вопросы

1. Верно ли утверждение, что процессы, в отличие от потоков, выполняются в од-ном и том же адресном пространстве?
2. В чем разница между процессом и потоком?
3. Каков принцип работы блочного устройства ввода-вывода?
4. Каков принцип работы символьного устройства ввода-вывода?
5. В чем заключается задача контроллера устройства?
6. Почему механизм прямого доступа к памяти повышает производительность компьютера?
7. Является ли обработчик прерывания частью драйвера устройства?
8. Каким образом можно выполнить программу, размер которой превышает объем доступной физической памяти?

Задания для самостоятельной работы